

WT EXPERIMENT - 04

Name	Class	Roll no
Ria Chaudhari	D15A	07
Eesha Chavan	D15A	08
Shraeyaa Dhaigude	D15A	15

Aim: Component Description and Arduino Working

Introduction:

This document aims to provide a detailed overview of the components used in a smart toll tax system and explain the Arduino code's functionality. The system uses RFID technology to automate toll collection by verifying RFID card IDs against authorized IDs, controlling a servo motor to open or close a toll barrier, and displaying the access status on an LCD. The focus will be on the components used and the Arduino code, excluding interfacing details.

Components Used

1. Arduino Uno: Serves as the central processing unit, reading data from the RFID reader, controlling the servo motor, and updating the LCD.
2. RFID Reader (RC522): Reads the unique identification (UID) from RFID cards or tags.
 - Communicates with the Arduino via SPI protocol.
3. RFID Cards/Tags: Each card contains a unique ID used for vehicle authorization.
4. Servo Motor: Controls the opening and closing of the toll barrier.
5. 16x2 I2C LCD Display: Displays access status messages, such as "Access Granted" or "Access Denied."
 - Communicates with the Arduino using I2C protocol.
6. Buzzer: Provides an audible signal for access confirmation or denial.
7. Jumper Wires and Breadboard: Used for making temporary circuit connections.

Arduino Uno Overview

The Arduino Uno is a widely used microcontroller board based on the ATmega328P. It is an open-source platform designed to be easy to use for beginners and flexible for advanced users. Key features include:

- Microcontroller: ATmega328P
- Operating Voltage: 5V

- Digital I/O Pins: 14 (6 can be used as PWM outputs)
- Analog Input Pins: 6
- Flash Memory: 32 KB
- Clock Speed: 16 MHz
- USB Connection: For programming and power
- Communication Protocols: Supports SPI, I2C, and UART

Arduino Code Explanation

The Arduino code for the smart toll system manages the RFID card reading, verification, servo motor control, and LCD display updates. Below is a breakdown of the code's functionality:

cpp

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#include <SPI.h>
```

```
#include <MFRC522.h>
```

```
#include <Servo.h>
```

```
#define SS_PIN 10
```

```
#define RST_PIN 9
```

```
#define BUZZER 2 // Buzzer pin
```

```
MFRC522 mfrc522(SS_PIN, RST_PIN); // RFID instance
```

```
Servo myServo; // Servo instance
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD instance with I2C address 0x27
```

```
String authorizedUID = "BD 31 15 2B"; // Example authorized UID
```

```
void setup() {
```

```
  Serial.begin(9600);
```

```
  SPI.begin();
```

```
  mfrc522.PCD_Init();
```

```
  myServo.attach(6);
```

```
  myServo.write(0); // Initial servo position
```

```
  pinMode(BUZZER, OUTPUT);
```

```
  noTone(BUZZER); // Ensure buzzer is off
```

```
  lcd.init();
```

```
  lcd.clear();
```

```
  lcd.backlight();
```

```
  lcd.setCursor(0, 0);
```

```
  lcd.print("Put card on");
```

```
  lcd.setCursor(0, 1);
```

```

    lcd.print("reader...");
}

void loop() {
    if (!mfrc522.PICC_IsNewCardPresent()) {
        return;
    }

    if (!mfrc522.PICC_ReadCardSerial()) {
        return;
    }

    String content = "";
    for (byte i = 0; i < mfrc522.uid.size; i++) {
        content.concat(String(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " "));
        content.concat(String(mfrc522.uid.uidByte[i], HEX));
    }
    content.trim();

    Serial.print("Card UID: ");
    Serial.println(content);

    lcd.clear();
    lcd.setCursor(0, 0);

    if (content == authorizedUID) {
        Serial.println("Authorized access");
        lcd.print("Access Granted");
        tone(BUZZER, 1000); // Play a tone
        myServo.write(90); // Open the toll barrier
        delay(3000);
        noTone(BUZZER); // Turn off the tone
        myServo.write(0); // Close the toll barrier
    } else {
        Serial.println("Access denied");
        lcd.print("Access Denied");
        tone(BUZZER, 500); // Different tone for denial
        delay(3000);
        noTone(BUZZER);
    }

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Put card on");
}

```

```

lcd.setCursor(0, 1);
lcd.print("reader...");

mfrc522.PICC_HaltA();
mfrc522.PCD_StopCrypto1();
}

```

Libraries and Definitions:

- Includes necessary libraries for LCD (Wire, LiquidCrystal_I2C), RFID (SPI, MFRC522), and servo motor control (Servo).
- Defines the pins for the RFID module (SS_PIN, RST_PIN) and the buzzer.

Global Object Initialization:

- Creates instances for the RFID reader (`mfrc522`), servo motor (`myServo`), and LCD (`lcd`).
- Defines `authorizedUID` to store the valid RFID card UID.

Setup Function:

- Initializes serial communication for debugging.
- Initializes the SPI bus and RFID reader.
- Attaches the servo motor to a digital pin and sets its initial position.
- Configures the buzzer pin as an output and ensures it is initially off.
- Initializes the LCD, clears it, turns on the backlight, and displays an initial message.

Loop Function:

- Checks for a new RFID card. If no card is present, the function exits early.
- Reads the UID of the detected card.
- The code retrieves the UID from the RFID card and converts it into a string format.
- Compares the read UID against a predefined authorized UID.
- If the UID is authorized, it displays "Access Granted" on the LCD, activates the buzzer, opens the toll barrier using the servo motor, pauses, deactivates the buzzer, and closes the barrier.
- If the UID is not authorized, it displays "Access Denied" on the LCD, activates the buzzer with a different tone, and pauses.
- Resets the LCD display to prompt for the next card.
- Halts the RFID card and stops the crypto functions to prevent data corruption.

Code Breakdown

1. Includes:

- `<Wire.h>`: For I2C communication (LCD).
 - `<LiquidCrystal_I2C.h>`: For controlling the LCD.
 - `<SPI.h>`: For SPI communication (RFID Reader).
 - `<MFRC522.h>`: For RFID module functionality.
 - `<Servo.h>`: For controlling the servo motor.
2. Definitions:
 - `SS_PIN` and `RST_PIN` define the pins connected to the RFID reader.
 - `BUZZER` defines the pin connected to the buzzer.
 3. Object Initialization:
 - `MFRC522 mfrc522(SS_PIN, RST_PIN)`: Creates an instance of the RFID reader.
 - `Servo myServo`: Creates an instance of the servo motor.
 - `LiquidCrystal_I2C lcd(0x27, 16, 2)`: Creates an instance of the LCD with the I2C address 0x27 (common address) and specifies a 16x2 display.
 4. `setup()` Function:
 - `Serial.begin(9600)`: Initializes serial communication for debugging.
 - `SPI.begin()`: Initializes the SPI bus.
 - `mfrc522.PCD_Init()`: Initializes the RFID reader.
 - `myServo.attach(6)`: Attaches the servo motor to digital pin 6.
 - `myServo.write(0)`: Sets the initial position of the servo motor to 0 degrees.
 - `pinMode(BUZZER, OUTPUT)`: Sets the buzzer pin as an output.
 - `noTone(BUZZER)`: Ensures the buzzer is off initially.
 - LCD Initialization: Initializes the LCD, clears it, turns on the backlight, and prints a message prompting the user to present the card.
 5. `loop()` Function:
 - Checks for a new card:
 - `if (!mfrc522.PICC_IsNewCardPresent()) { return; }`
 - `if (!mfrc522.PICC_ReadCardSerial()) { return; }`
 - These lines check if a new card is present and if its serial data can be read. If not, the loop restarts.
 - Reads the card's UID:
 - The code reads the UID from the RFID card and converts it into a string format.
 - Access Control Logic:
 - The code checks if the card's UID matches a pre-specified value (`authorizedUID`). If it matches, it prints "Authorized access" to the serial monitor and LCD, activates the buzzer, and opens the toll gate by moving the servo to 90 degrees. After a 3-second delay, it deactivates the buzzer and closes the gate by moving the servo back to 0 degrees.
 - If the UID does not match, it prints "Access denied" to the serial monitor and LCD and activates the buzzer with a different tone.

- LCD Update:
 - The LCD is cleared, and a new message is displayed, prompting the user to present the card again.
- Card Halt:
 - `mfrc522.PICC_HaltA()`: Halts the RFID card.
 - `mfrc522.PCD_StopCrypto1()`: Stops the crypto functions on the MFRC522 reader.

Conclusion:

This document detailed the components and Arduino code used in a smart toll tax system. The system automates toll collection by integrating an RFID reader for card identification, a servo motor for gate control, and an LCD for displaying the access status. The Arduino code manages these components, providing an efficient and automated toll collection process.